Morphic Studio

Document 15 — Asset Library (Expanded Specification)

Document ID: MS-015

Version: 2.0

Status: Expanded Specification

Priority: Critical

Purpose

The Asset Library is the permanent repository for every approved creative asset within Morphic Studio.

Unlike traditional file storage systems, the Asset Library is intelligent. It understands relationships between assets, tracks their history, organizes them by context, and enables AI systems to reuse existing assets instead of recreating them.

Every image, animation, voice, sound effect, background, prop, outfit, expression, storyboard, and generated element should become part of the creator's reusable creative ecosystem.

The Asset Library embodies one of Morphic Studio's core philosophies:

> **Generate Once. Save Permanently. Reuse Forever.**

Vision

Creators should never lose work or regenerate something they have already approved unless they intentionally request a new variation.

The Asset Library should become a growing creative vault that increases in value over time.

As projects grow, the library becomes smarter, faster, and more useful because AI can reuse previously approved assets while maintaining consistency.

Core Principles

The Asset Library shall be:

- Permanent
- Searchable
- Reusable
- Versioned
- AI-aware
- Organized
- Expandable
- Secure
- Fast
- Creator-controlled

Objectives

The Asset Library shall:

- Store all approved assets.
- Eliminate unnecessary regeneration.
- Improve AI consistency.
- Reduce production costs.
- Increase generation speed.
- Preserve creative history.
- Enable future collaboration.
- Support franchise-scale projects.

What Is an Asset?

In Morphic Studio, an asset is **any reusable creative element**.

Examples include:

Visual Assets

- Characters
- Character Variations
- Outfits
- Hairstyles
- Facial Expressions
- Hand Poses

- Body Poses
- Backgrounds
- Props
- Vehicles
- Buildings
- Effects
- Lighting Presets
- Camera Angles

Story Assets

- Story Bible Entries
- Character Notes
- World Rules
- Timeline Events
- Episode Summaries
- Dialogue Templates
- Scene Templates

Animation Assets

- Walk Cycles
- Run Cycles
- Idle Animations
- Fight Animations
- Camera Movements
- Scene Transitions
- Motion Presets
- Lip-Sync Data
- Facial Animation Presets

Audio Assets

- Voice Profiles
- Music Tracks
- Sound Effects
- Ambient Sounds
- Environmental Audio
- Voice Emotions
- Character Voices

AI Assets

- Prompt Templates
- AI Planning Templates
- Workflow Presets
- Generation Settings
- Style Profiles
- Reference Images
- Control Images

Asset Categories

Each asset belongs to one or more categories.

Examples:

- Character
- Clothing
- Location
- Prop
- Environment
- Audio
- Animation
- Camera
- Script
- Story
- Effect
- User Upload
- AI Generated

Categories make searching and filtering easier.

Asset Metadata

Every asset should store metadata such as:

- Asset ID
- Name
- Description
- Creator
- Project
- Asset Type
- Category

- Creation Date
- Last Modified
- Tags
- Related Characters
- Related Locations
- Related Stories
- Current Version
- Previous Versions
- Approval Status
- Source (AI, Upload, Imported)

This metadata enables efficient management and retrieval.

Version Control

Every asset supports version history.

Creators can:

- View previous versions.
- Restore older versions.
- Compare versions.
- Duplicate assets.
- Branch assets into variations.

No approved asset is permanently overwritten.

AI Integration

Before generating new content, Morphic Core shall:

1. Search the Asset Library.
2. Check for matching approved assets.
3. Determine whether reuse is appropriate.
4. Recommend existing assets.
5. Generate only what is missing.
6. Save newly approved assets automatically.

The AI should prioritize reuse over regeneration whenever possible.

Search System

Creators should be able to search using:

- Asset Name
- Tags
- Character
- Location
- Scene
- Episode
- Color
- Style
- Asset Type
- Date
- Creator
- AI Status

Advanced filters should allow complex searches.

Asset Relationships

Assets should understand how they connect.

Example:

John Character

↓

School Uniform

↓

School Classroom

↓

School Backpack

↓

Math Textbook

↓

Chapter 3

↓

Episode 5

Rather than isolated files, the Asset Library maintains a connected creative graph.

User Stories

As a creator, I want to reuse a previously approved background instead of generating a new one.

As a creator, I want every outfit belonging to a character organized together.

As a creator, I want AI to automatically recommend assets I already own.

As a creator, I want to search my entire creative library instantly.

As a creator, I want to organize assets without manually managing folders.

Functional Requirements

The Asset Library shall allow users to:

- Upload assets.
- Import assets.
- Generate assets with AI.
- Rename assets.
- Tag assets.
- Archive assets.
- Delete assets.
- Restore archived assets.
- Duplicate assets.
- Create collections.
- Share assets (Future).
- Export assets.
- Preview assets.

Non-Functional Requirements

The Asset Library should:

- Scale to millions of assets.
- Return search results quickly.
- Synchronize automatically.
- Cache frequently used assets.
- Support cloud storage.
- Support future offline synchronization.
- Remain responsive during AI generation.

Database Entities

Core entities include:

- Asset
- AssetVersion
- AssetCollection
- AssetTag
- AssetRelationship
- AssetUsage
- AssetHistory
- AssetPermission
- AssetMetadata

API Considerations

Potential endpoints include:

- Create Asset
- Retrieve Asset
- Update Asset
- Delete Asset
- Search Assets
- List Asset Versions
- Duplicate Asset
- Archive Asset
- Restore Asset
- Export Asset

Error Handling

If duplicate assets are detected:

- Suggest reuse.
- Display similarity score.
- Allow the creator to decide.

If uploads fail:

- Preserve progress.
- Retry automatically.
- Display clear error messages.

Future Expansion

Future versions may support:

- Cross-project asset sharing.
- Team asset libraries.
- AI-generated asset recommendations.
- Automatic duplicate detection.
- Marketplace integration.
- Plugin-generated assets.
- Community asset packs.
- Franchise-wide shared libraries.

Acceptance Criteria

The Asset Library succeeds when creators can instantly locate, reuse, and manage every creative asset without unnecessary duplication while AI consistently builds upon previously approved work.

Key Principle

Every approved asset becomes part of the creator's permanent creative memory. The library should grow more valuable with every project, never forcing creators to start from scratch.

End of Document 15